Solution: Generate Binary Numbers from 1 to N

Problem Statement

Given an integer `N`, generate all binary numbers from 1 to N and return them as a list of strings.

Examples:

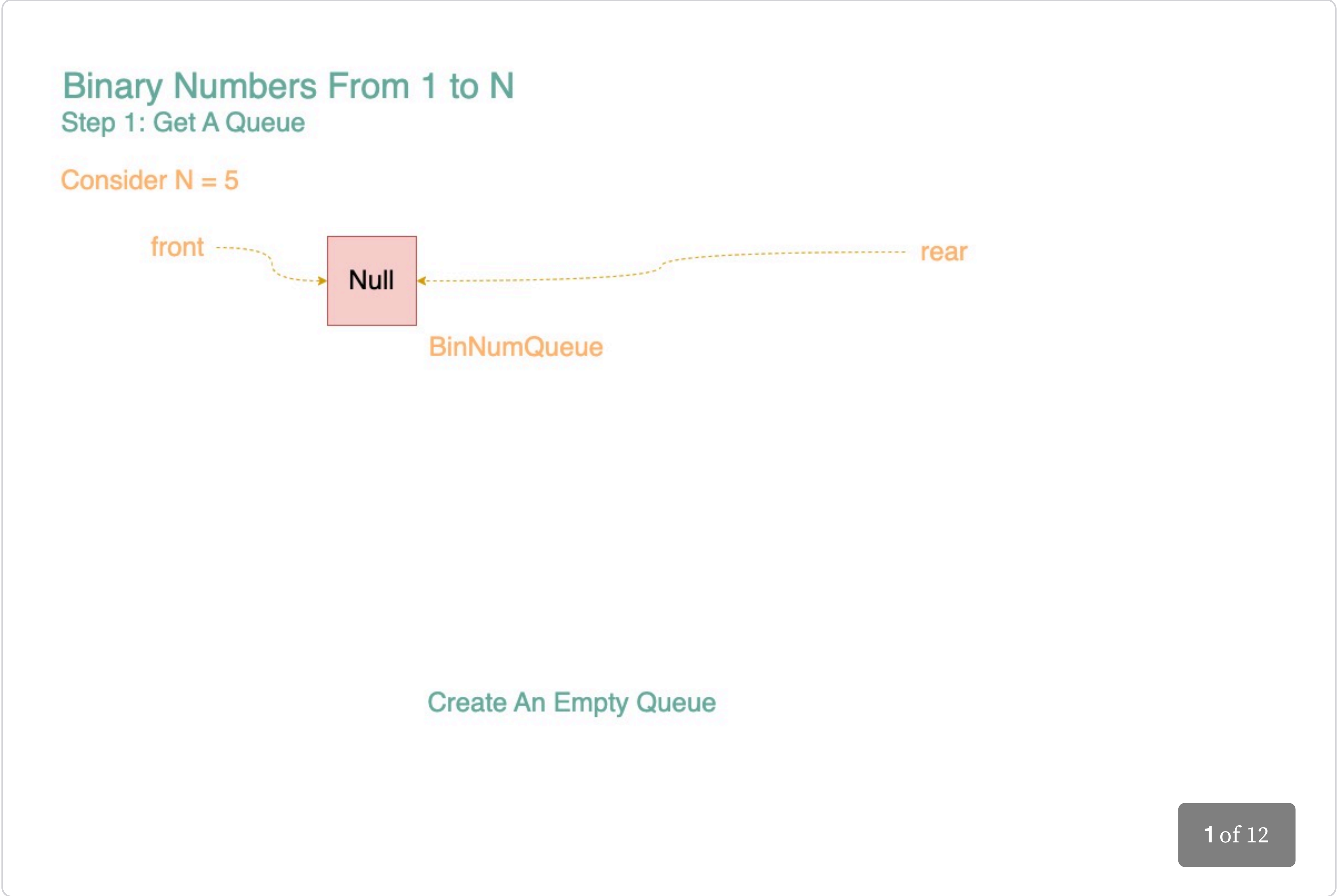
1. **Input**: N = 2
   Output: ["1", "10"]
   Explanation: The binary representation of 1 is "1", and the binary representation of 2 is "10".
2. **Input**: N = 3
   Output: ["1", "10", "11"]
   Explanation: The binary representation of 1 is "1", the binary representation of 2 is "10", and the binary representation of 3 is "11".
3. **Input**: N = 5
   Output: ["1", "10", "11", "100", "101"]
   Explanation: These are the binary representations of the numbers from 1 to 5.

Solution

To solve this problem, we'll use a queue to systematically generate binary numbers. Initially, we enqueue the binary representation of '1'. For each number from 1 to N, we perform the following steps: dequeue the front element of the queue and record it as the current binary number. Then, we generate the next two binary numbers by appending '0' and '1' to the current number and enqueue these new numbers. This process continues until we have generated all binary numbers up to N. This method efficiently leverages the queue's FIFO (First In First Out) nature to build upon previous binary numbers, ensuring a systematic and orderly generation of binary representations.

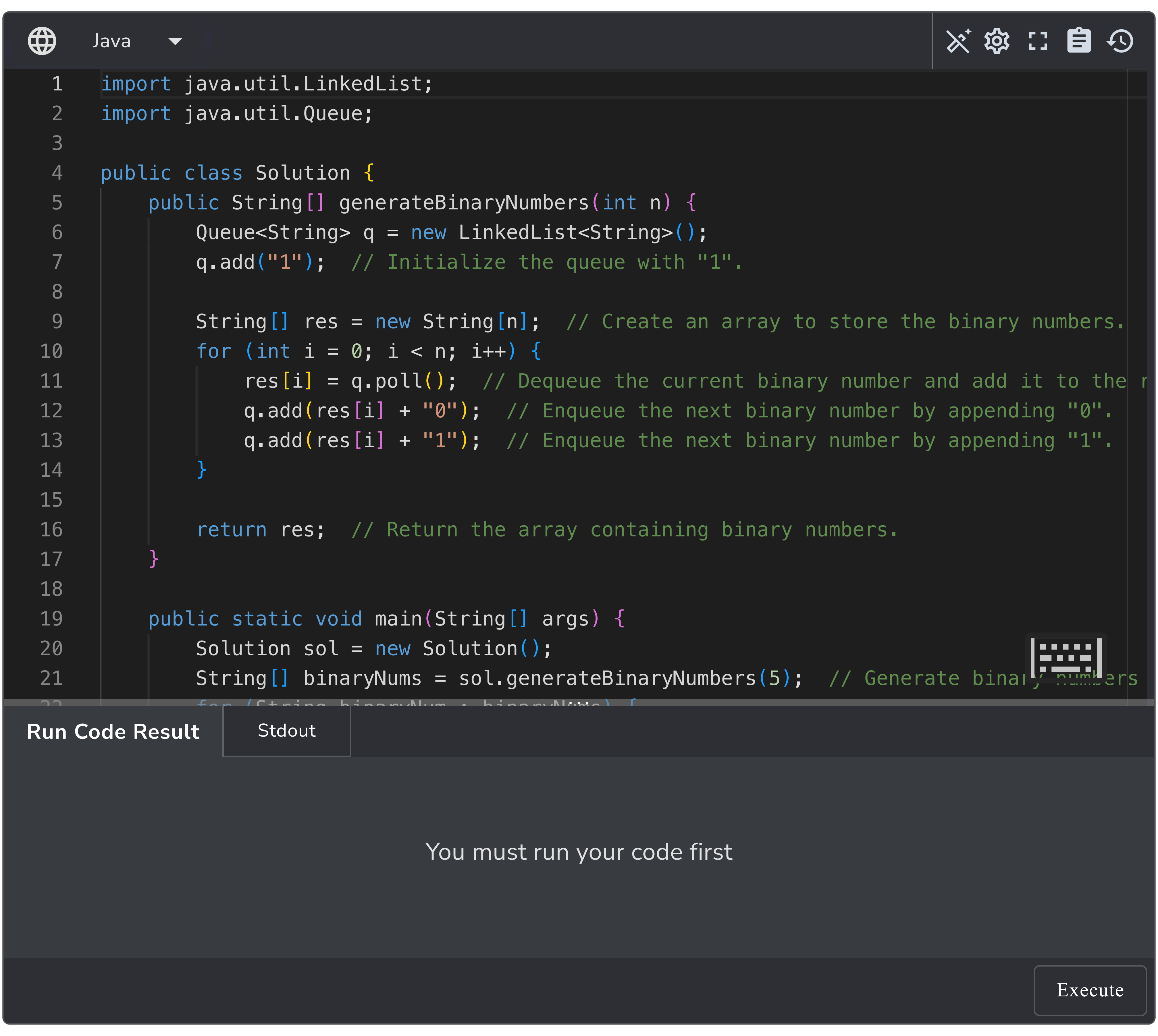
Step-by-Step Solution

1. **Initialize a Queue:** Create a queue data structure which will be used to hold binary numbers in string format.
2. **Start with '1':** Enqueue the binary representation of the first number, which is '1'.
3. **Iterate up to N:** Set up a loop that runs from 1 to N. This loop controls how many binary numbers you need to generate.
4. **Dequeue and Output:** In each iteration of the loop, dequeue an element from the front of the queue. This element is the binary representation of the current number. Store or print this number as part of the solution.
5. **Generate Next Binary Numbers:**
   - Take the dequeued binary number and append '0' to it, forming the next binary number. Enqueue this new number back into the queue.
   - Repeat the above step, but this time append '1' instead of '0'.
6. **Repeat the Process:** Continue this process until the loop completes its iteration up to N. Each iteration generates the next set of binary numbers based on the current numbers in the queue.



Code

Here is how we can implement this algorithm:



```java
import java.util.LinkedList;
import java.util.Queue;

public class Solution {
    public String[] generateBinaryNumbers(int n) {
        Queue<String> q = new LinkedList<String>();
        q.add("1");  // Initialize the queue with "1".

        String[] res = new String[n];  // Create an array to store the binary numbers.
        for (int i = 0; i < n; i++) {
            res[i] = q.poll();  // Dequeue the current binary number and add it to the r
            q.add(res[i] + "0");  // Enqueue the next binary number by appending "0".
            q.add(res[i] + "1");  // Enqueue the next binary number by appending "1".
        }

        return res;  // Return the array containing binary numbers.
    }

    public static void main(String[] args) {
        Solution sol = new Solution();
        String[] binaryNums = sol.generateBinaryNumbers(5);  // Generate binary numbers
```

You must run your code first

Time and Space Complexity

The time complexity of this algorithm is O(N) because we're processing N binary numbers. The space complexity is also O(N) because we're storing N binary numbers in the queue and the result list.

← Previous
Problem 3: Generate Binary Numbers From 1 To N

Next →
Problem 4: Palindrome Check Using Queue

☐ Mark as Completed